

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

Y.Ganesh Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

ANSWERS: -

Analysis of a Smart City Surveillance System

A smart city surveillance system continuously collects video streams from multiple cameras. These video feeds are transferred from I/O devices to memory and then processed by the CPU for real-time threat detection. During peak hours, the large amount of data creates communication delays between the CPU, memory, and I/O devices, reducing overall system performance.

1. Functional Units of a Computer System

a) Central Processing Unit (CPU)

- Executes instructions and controls system operations.
- Consists of:
 - **Arithmetic Logic Unit (ALU):** Performs arithmetic and logical operations.
 - **Control Unit (CU):** Controls instruction execution and coordinates data transfer.
 - **Registers:** Store temporary data and instructions for fast access.

Role in Surveillance System:

- Processes image frames.
- Runs AI algorithms for object detection and threat identification.
- Makes real-time decisions.

b) Memory Unit

- Stores instructions, operating system, and video data.
- Types:
 - **Cache Memory:** Fastest memory used for frequently accessed data.
 - **RAM:** Temporarily stores video frames and processing data.
 - **Secondary Storage (SSD/HDD):** Stores recorded surveillance footage.

Problem During Peak Hours:

- Large video streams quickly occupy RAM.
- Frequent memory accesses increase waiting time for the CPU.

c) Input/Output (I/O) Devices

- **Input:** CCTV cameras, motion sensors, microphones.
- **Output:** Monitors, alarms, notification systems.

The I/O subsystem transfers huge amounts of video data to memory through system buses.

d) Bus Structure

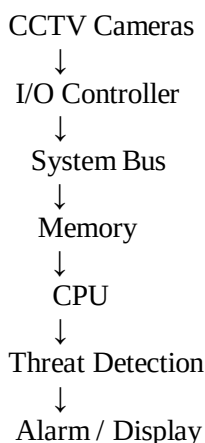
A bus connects CPU, memory, and I/O devices.

Types of Buses

- Address Bus
- Data Bus
- Control Bus

2. Interaction Between CPU, Memory, and I/O

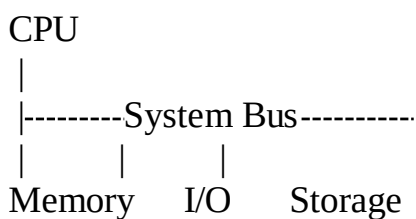
Data Flow



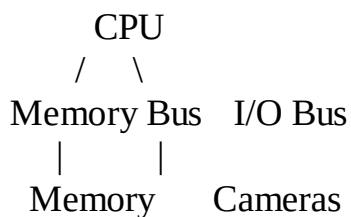
3. Single-Bus vs Multi-Bus Structure

Feature	Single Bus	Multi-Bus
Number of buses	One	Multiple
Data transfer	One transfer at a time	Multiple simultaneous transfers
CPU waiting time	High	Low
Bus congestion	High	Low
Cost	Low	Higher
Performance	Moderate	High

Single-Bus Architecture



Multi-Bus Architecture



4. Instruction Execution Cycle

The CPU executes each instruction in four stages.

Step 1: Fetch

- Retrieve instruction from memory.

Step 2: Decode

- Control Unit interprets the instruction.

Step 3: Execute

- ALU performs computation.

Step 4: Store (Write Back)

- Store the result in registers or memory.

5. Improvements in Instruction Execution Cycle

a) Instruction Pipelining

- Executes multiple instruction stages simultaneously.
- Increases CPU utilization.
- Reduces processing time.

Example:

Clock	Instruction 1	Instruction 2	Instruction 3
1	Fetch		
2	Decode	Fetch	
3	Execute	Decode	Fetch
4	Store	Execute	Decode

b) Superscalar Processing

- Executes multiple instructions in one clock cycle.
- Benefit:
- Faster image processing.

c) Cache Memory Optimization

- Frequently used data stored in cache.
- Reduces RAM access time.

d) DMA (Direct Memory Access)

- I/O devices transfer data directly to memory without CPU involvement.
- CPU becomes available for processing video analytics.

6. Addressing Modes

Addressing modes determine how operands are accessed.

Addressing Mode	Description	Example
Immediate	Operand is part of instruction	MOV R1,#10
Direct	Address specified directly	LOAD 5000
Indirect	Address stored in another location	LOAD (R2)
Register	Operand stored in register	ADD R1,R2
Indexed	Base address + index	Video_Frame[i]

e) Benchmarking

Benchmarking measures system performance using standard workloads.
For surveillance systems, benchmarks evaluate:

- Frame processing rate (FPS)
 - CPU utilization
 - Memory bandwidth
 - I/O throughput
 - Threat detection latency
- Higher benchmark scores indicate better overall performance.

8. Methods to Improve System Performance

- Replace a single-bus architecture with a multi-bus architecture to reduce bus contention.
- Increase RAM capacity and use faster cache memory to improve memory access speed.

- Implement DMA for direct data transfer between cameras and memory.
- Use instruction pipelining and superscalar execution to process more instructions per clock cycle.
- Optimize addressing modes (such as indexed addressing) for efficient video frame access.
- Employ multicore processors to analyze multiple camera streams in parallel.
- Monitor CPI, clock cycles, execution time, and benchmarking results to identify and eliminate performance bottlenecks.

2 answer:-

Analysis of a Real-Time Stock Trading Application

A real-time stock trading application executes millions of instructions every second to process market data, execute buy/sell orders, and update user accounts. During periods of high market activity, users experience delays because the processor has a **high Cycles Per Instruction (CPI)** caused by frequent memory accesses and branch instructions. These factors increase CPU execution time and reduce overall system performance.

1. Fetch–Decode–Execute Cycle

The CPU processes every instruction through three main stages.

Step 1: Fetch

- The CPU fetches the next instruction from memory.
- The Program Counter (PC) stores the address of the next instruction.

Issue:

- Frequent memory accesses increase fetch time.
- Cache misses force the CPU to wait for slower main memory.

Step 2: Decode

- The Control Unit interprets the instruction.
- Determines the required operation and operands.

Issue:

- Complex branch instructions require additional decoding.
- Incorrect branch predictions waste CPU cycles.

Step 3: Execute

- The ALU performs arithmetic or logical operations.
- Results are written back to registers or memory.

Issue:

- Memory-intensive instructions delay execution.
- CPU stalls while waiting for data from memory.

Instruction Cycle

Fetch



Decode



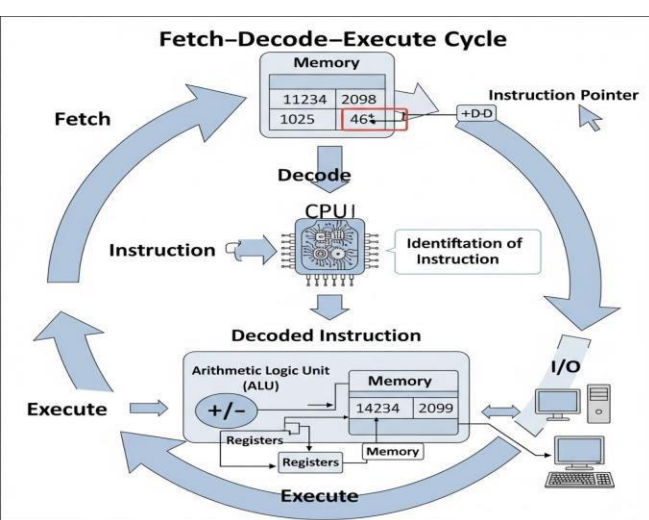
Execute



Write Back



Next Instruction



3. Causes of High CPI

a) Frequent Memory Access

- Large financial datasets are stored in RAM.
- Cache misses require slow memory access.
- CPU waits during data retrieval.

b) Branch Instructions

Examples:

- Buy/Sell decision
- Risk verification
- Price comparison
- Fraud detection

Frequent conditional branches create pipeline stalls and increase CPI.

c) Cache Misses

- Requested data is unavailable in cache.
- CPU accesses slower main memory.
- Execution time increases.

d) Pipeline Stalls

Waiting for:

- Memory access
- Branch resolution

- Data dependencies

These stalls increase idle CPU cycles.

4. Methods to Reduce CPI

a) Increase Cache Memory

- Larger L1/L2/L3 cache
- Higher cache hit rate
- Faster instruction fetch

Benefit

- Reduced memory access time
- Lower CPI

b) Instruction Pipelining

Pipeline stages overlap.

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	Fetch		
2	Decode	Fetch	
3	Execute	Decode	Fetch
4	Write Back	Execute	Decode

e) Out-of-Order Execution

Independent instructions execute before waiting instructions.

Benefits:

- Better CPU utilization.
- Fewer idle cycles.

f) Prefetching

The CPU loads future instructions into cache before they are needed.

Benefits:

- Faster fetch stage.
- Lower latency.

g) Better Memory Hierarchy

Use:

- Faster cache memory
- High-speed DDR RAM
- Solid-State Drives (SSD)

Benefits:

- Reduced memory waiting time.
- Faster transaction processing.

6. Architectural Enhancements

Enhancement	Purpose	Performance Improvement
Multi-level Cache	Reduce memory access	Lower CPI
Branch Prediction	Reduce branch stalls	Faster execution
Instruction Pipeline	Overlap execution stages	Higher throughput
Superscalar CPU	Execute multiple instructions simultaneously	More instructions per cycle
Out-of-Order Execution	Reduce idle cycles	Improved CPU utilization
Speculative Execution	Execute predicted instructions early	Faster branch handling
Multi-Core Processor	Parallel transaction processing	Increased system throughput

Overall Improvement Strategy

- Increase cache size to reduce memory access delays.
- Use instruction pipelining to overlap fetch, decode, and execute stages.
- Implement branch prediction to minimize pipeline stalls.
- Adopt superscalar and out-of-order execution for parallel instruction processing.
- Use faster memory and optimized memory hierarchy.
- Employ multi-core processors to process multiple trading requests simultaneously.
- Continuously monitor CPI, clock cycles, throughput, and benchmark results to optimize CPU performance.

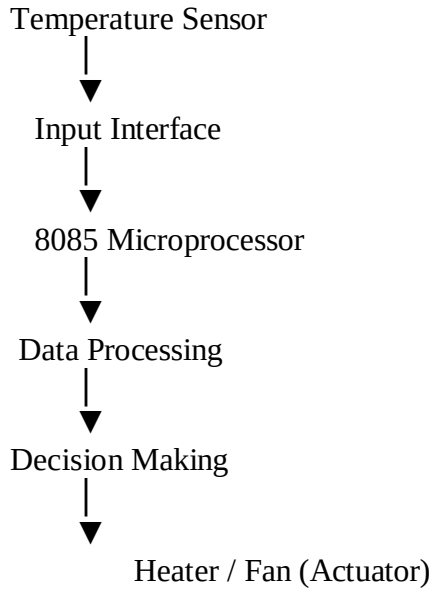
3 answer: -

Analysis of an 8085-Based Industrial Temperature Control Unit

An industrial temperature control unit uses an **8085 microprocessor** to read temperature data from sensors, process the values, and control actuators such as heaters or cooling fans.

Incorrect temperature readings may occur due to improper use of **addressing modes**, causing the processor to access incorrect memory locations or data. Proper use of the **8085 instruction set** ensures accurate sensor data processing and reliable system operation.

1. Working of the Embedded Temperature Control System



. Addressing Modes of 8085

Addressing modes determine how the processor accesses operands during instruction execution.

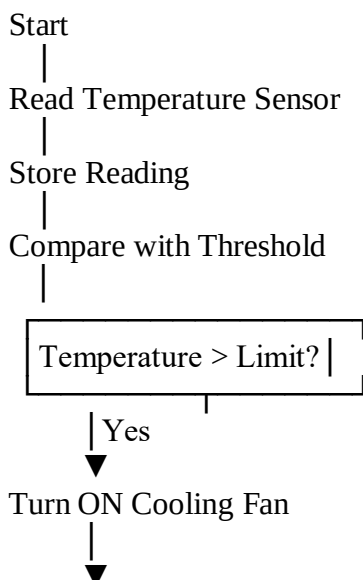
a) Immediate Addressing Mode

The data is directly included in the instruction.

3. Influence of Addressing Modes on Data Access

Addressing Mode	Data Access	Effect
Immediate	Data inside instruction	Fast execution
Register	Data in registers	Very fast
Direct	Fixed memory address	Suitable for constant data
Register Indirect	Memory through register pair	Flexible for sensor arrays
Implied	Operand implied	No explicit operand required

Example Program Flow



No
|
Turn ON Heater
|
▼
Repeat

Methods to Ensure Accurate and Reliable Operation

- Use the correct addressing mode for each type of data access.
- Initialize registers and memory before processing.
- Verify register pair values before indirect memory access.
- Store calibration values at fixed memory locations.
- Validate sensor readings using comparison instructions (CMP).
- Avoid overwriting registers containing sensor data.
- Use conditional branching (JZ, JC, JNZ) for accurate decision making.
- Test and debug the program thoroughly to eliminate addressing errors.

Advantages of Proper Addressing Modes

- Faster program execution.
- Accurate sensor data acquisition.
- Reduced programming errors.
- Reliable temperature monitoring.
- Improved actuator control.
- Better industrial system safety.

4th Answer: -

4. Evaluation of 8086 Segmented Memory and Program Execution

An 8086 microprocessor uses segmented memory architecture to efficiently organize memory and execute multiple programs. Memory is divided into Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). If segment registers or addressing modes are used incorrectly, problems such as incorrect data access, stack overflow, and program crashes can occur.

1. Segmented Memory Architecture of 8086

The 8086 has a 20-bit address bus, allowing it to access 1 MB of memory. Since its registers are only 16 bits, it uses segmentation to generate the physical address.

Physical Address Formula

$$\text{Physical Address} = (\text{Segment Address} \times 10H) + \text{Offset Address}$$

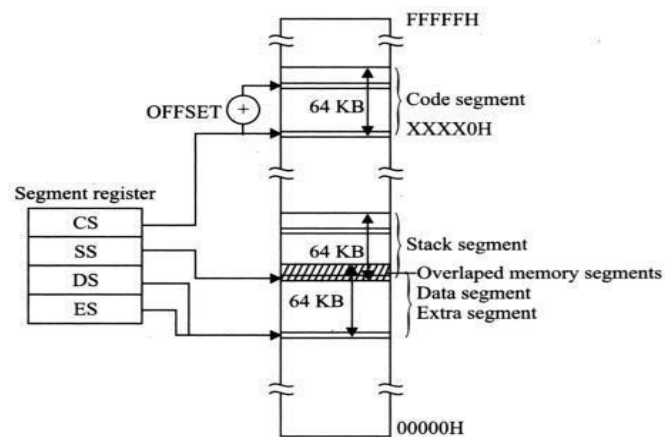
Example:

- Segment = 2000H
- Offset = 1200H

$$\text{Physical Address} = 20000H + 1200H = 21200H$$

2. Types of Memory Segments

Segment	Register	Purpose
Code Segment	CS	Stores executable instructions
Data Segment	DS	Stores variables and data
Stack Segment	SS	Stores return addresses and local data
Extra Segment	ES	Stores additional data for string operations



The segmented memory.

4. Working of Segmentation

Code Segment (CS)

Stores program instructions.

Example:

MOV AX, BX

ADD AX, CX

The Instruction Pointer (IP) contains the offset of the next instruction.

Physical Address:

$$CS \times 10H + IP$$

Data Segment (DS)

Stores variables and program data.

Example:

MOV AX, [1000H]

The processor accesses memory through the DS register.

Stack Segment (SS)

Used for:

- Function calls
- Interrupt handling
- Temporary storage
- Local variables

Instructions:

PUSH AX

POP AX

CALL

RET

Stack Pointer (SP) always points to the top of the stack.

Extra Segment (ES)

Primarily used in string operations.

Example:

MOVSB

MOVSW

5. Problems Due to Improper Segment Handling

a) Incorrect Data Access

If DS points to the wrong segment,

Example:

Instead of

DS = 3000H

Program accidentally loads

DS = 4000H

Result:

- Wrong memory location accessed
 - Incorrect data processing
 - Invalid program output
-

b) Stack Overflow

Occurs when excessive PUSH instructions fill the stack beyond its allocated space.

Example:

PUSH AX

PUSH BX

PUSH CX

...

without corresponding POP instructions.

Effects:

- Memory corruption
 - Program crash
 - Loss of return addresses
-

c) Incorrect Code Execution

If CS changes accidentally,

The processor executes unintended instructions.

Result:

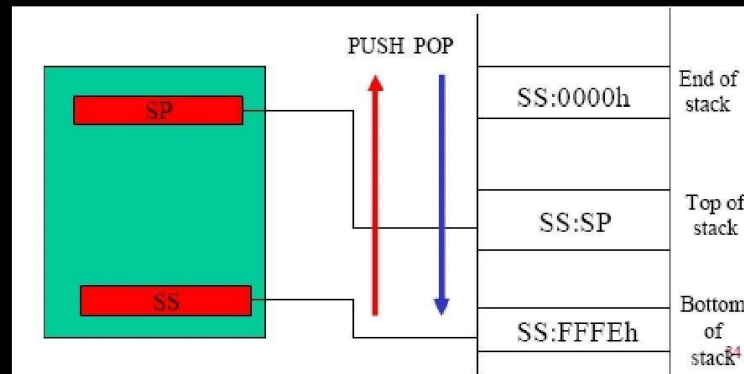
- Unexpected behavior
 - System failure
-

d) Wrong Offset Address

Incorrect offset values cause access to invalid memory locations.

6. Stack Memory Operation

The Stack



11

7. Addressing Modes in 8086

Addressing Mode	Description	Example
Immediate	Operand is part of the instruction	MOV AX,1000H
Register	Operand in register	MOV AX,BX
Direct	Memory address given directly	MOV AX,[2000H]
Register Indirect	Address stored in register	MOV AX,[BX]
Based	Base register used	MOV AX,[BX+10]
Indexed	Index register used	MOV AX,[SI]
Based Indexed	Base + Index	MOV AX,[BX+SI]
Based Indexed with Displacement	Base + Index + Offset	MOV AX,[BX+SI+20]

Ensuring Correct Program Execution

- Initialize **CS**, **DS**, **SS**, and **ES** correctly before execution.
- Use the appropriate addressing mode for each memory access.
- Balance every **PUSH** with a corresponding **POP** instruction.
- Avoid modifying segment registers unintentionally.
- Validate memory addresses before accessing data.
- Allocate sufficient stack space to prevent overflow.
- Use **CALL** and **RET** instructions correctly for subroutine handling.
- Test and debug programs to identify segmentation and addressing errors.

10. Advantages of Proper Segment Management

- Efficient utilization of the 1 MB memory space.
- Faster execution through organized code and data storage.
- Prevention of stack overflow and memory corruption.
- Accurate data access and processing.
- Improved reliability for multitasking applications.
- Simplified debugging and maintenance.

5. Analysis of Number System Conversion in a Data Communication System

A **data communication system** represents packet information in **hexadecimal (Hex)** format because it is compact, easy to read, and closely corresponds to binary data used by computers. However, if a software engineer incorrectly converts hexadecimal values to binary, packet information may be interpreted incorrectly, leading to transmission errors, incorrect instruction execution, and system failures. Efficient CPU design and benchmarking help detect and minimize such issues in real-time communication systems.

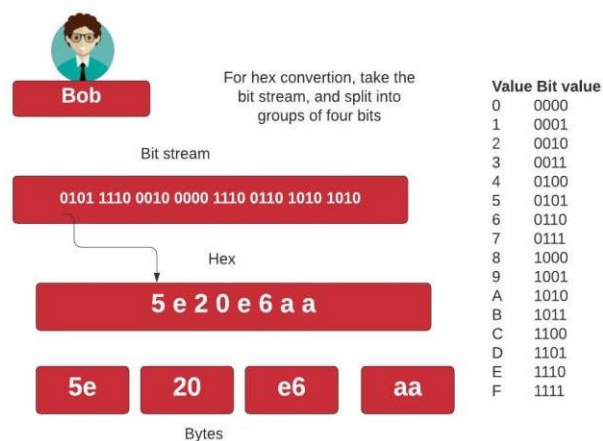
1. Number Systems Used in Data Communication

Number System	Base	Digits Used
Binary	2	0, 1
Decimal	10	0–9
Hexadecimal	16	0–9, A–F

2. Importance of Hexadecimal to Binary Conversion

Hex	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

3. Packet Representation



A communication packet typically consists of:

- Header
- Source Address
- Destination Address
- Payload
- Checksum

Hexadecimal values are converted into binary before processing by the CPU.

4. Importance of Number System Conversion

a) Easy Debugging

- Hexadecimal values are shorter and easier to read than binary.

Example:

Binary : 1101011010011110

Hex : D69E

b) Memory Address Repres

Example:

Memory Address = 1F2AH

instead of

0001111100101010₂

c) Packet Transmission

Communication protocols (Ethernet, TCP/IP, CAN) commonly display packet contents in hexadecimal format.

d) Instruction Representation

Machine instructions are stored in binary but displayed in hexadecimal for easier analysis.

5. Errors Due to Incorrect Conversion

a) Incorrect Data Interpretation

Suppose

Hex = 2F

Correct Binary

00101111

If converted incorrectly as

00111111

The received value changes completely.

b) Wrong Instruction Execution

Machine instructions are binary.

Incorrect conversion may generate an invalid instruction.

Result:

- Incorrect execution
- Program crash
- Unexpected output

c) Incorrect Memory Address

Example

Correct Address

4A20H

Incorrectly interpreted as

4B20H

CPU accesses the wrong memory location.

d) Packet Corruption

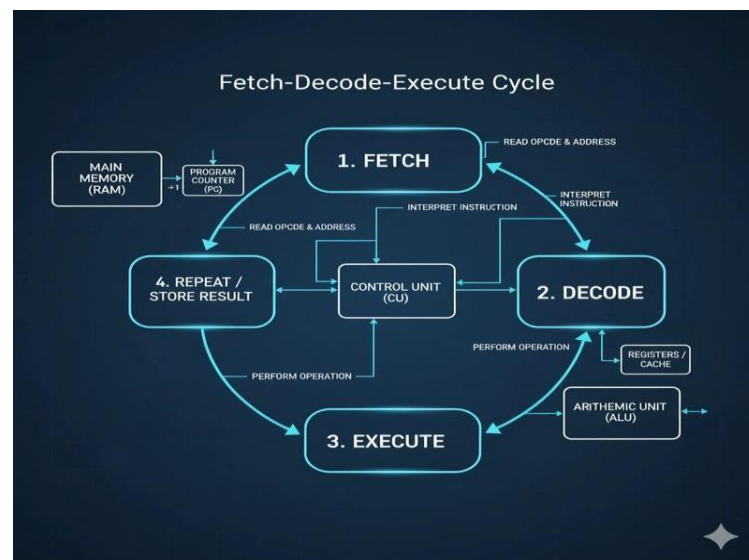
Wrong binary conversion changes:

- Packet header
- Payload
- Checksum

Result:

- Transmission failure
- Packet rejection

6. Instruction Execution Process



The CPU executes instructions through the following stages:

Step 1 – Fetch

- Read instruction from memory.

↓

Step 2 – Decode

- Convert binary instruction into CPU operations.

↓

Step 3 – Execute

- Perform arithmetic or logical operation.

↓

Step 4 – Store

- Write result to registers or memory.
- Incorrect binary conversion affects the decode stage, causing improper instruction execution.
-

7. Efficient CPU Design to Reduce Errors

a) Error Detection

- CPU's use:
- Parity checking
 - Error Correcting Code (ECC)
 - Checksum verification
- to identify incorrect data.
-

b) Pipeline Verification

Modern processors validate instruction integrity before execution.

c) Cache Memory

Frequently used instructions remain in cache, reducing repeated conversions and memory access delays.

d) High-Speed ALU

Performs binary operations accurately and efficiently.

e) Memory Protection

Prevents unauthorized access to invalid memory locations.

8. Benchmarking

Benchmarking measures system performance and reliability.
Common metrics include:

Metric	Purpose
CPI (Cycles Per Instruction)	CPU efficiency
Clock Rate	Processing speed
Throughput	Packets processed per second
Latency	Time taken to process a packet
Error Rate	Number of packet conversion errors
Memory Bandwidth	Data transfer speed

9. Methods to Minimize Conversion Errors

- Use reliable hexadecimal-to-binary conversion algorithms.
 - Validate packet headers and checksums before processing.
 - Implement parity and ECC for error detection and correction.
 - Use standardized packet formats and communication protocols.
 - Verify memory addresses before execution.
 - Employ CPU benchmarking to identify bottlenecks.
 - Test software thoroughly using debugging tools and simulation.
-

Advantages of Efficient CPU Design

- Faster instruction execution.
- Accurate packet decoding.
- Reduced communication errors.
- Improved real-time performance.
- Increased system reliability.